

山东大学



网络安全创新创业实践

——基于 libsecp256k1 的 ECDSA 自选摘要签名构造复现
与实现安全及性能分析

7.20 作业二

姓名	学号	班级
钟晴	202322460127	密码 2 班
宋坤鹏	202322460130	密码 1 班
赵思鹏	202300460075	密码 1 班
田学琛	202300460064	密码 1 班

山东大学网络空间安全学院

目录

1 实验要求	1
1.1 实验背景	1
1.2 实验题目	2
2 理论基础	3
2.1 secp256k1 与 ECDSA 签名验证原理	3
2.2 自选摘要签名构造及正确性分析	5
2.3 构造的适用范围与 lower-S 处理	6
3 实验环境与程序设计	7
3.1 实验环境与版本安排	7
3.2 总体流程与控制变量	8
3.3 程序设计与判定方法	9
4 ECDSA 自选摘要签名构造实验	10
4.1 官方库编译与基线验证	10
4.2 构造程序设计与实现	12
4.3 实验结果与分析	17
5 libsecp256k1 验证源码与安全修复分析	18
5.1 ECDSA 验证接口与内部计算	18
5.2 Jacobian 坐标下的横坐标判定	21
5.3 常数时间问题与修复	24
6 ECDSA 验证性能改进实验	27
6.1 性能改进内容与实验设计	27
6.2 测试结果与性能计算	30
6.3 性能变化原因及实验局限	33
7 实验总结	34
8 参考文献	36

1 实验要求

1.1 实验背景

本次实验的题目与主要要求如图 1.1 所示。实验首先要求分析一种特殊的 ECDSA 签名构造场景：当验证者只检查外部提供的消息摘要，而没有自行确认该摘要是否由真实消息计算得到时，攻击者可以在不知道私钥的情况下，构造一组能够通过 ECDSA 验证的摘要与签名。除此之外，还需要结合 Bitcoin Core 官方 libsecp256k1 仓库，分析密码算法实现中的安全修复和性能改进，并解释相关代码修改背后的数学与密码工程原理。

ECDSA – Forge signature when the signed message is not checked

- Key Gen: $P = dG$, n is order
 - Sign(m)
 - $k \leftarrow \mathbb{Z}_n^*$, $R = kG$
 - $r = R_x \bmod n, r \neq 0$
 - $e = \text{hash}(m)$
 - $s = k^{-1}(e + dr) \bmod n$
 - Signature is (r, s)
 - Verify (r, s) of m with P
 - $e = \text{hash}(m)$
 - $w = s^{-1} \bmod n$
 - $(r', s') = e \cdot wG + r \cdot wP$
 - Check if $r' = r$
 - Holds for correct sig since
 - $es^{-1}G + rs^{-1}P = s^{-1}(eG + rP) =$
 - $k(e + dr)^{-1}(e + dr)G = kG = R$
 - $\sigma = (r, s)$ is valid signature of m with secret key d
 - If only the hash of the signed message is required
 - Then anyone can forge signature $\sigma' = (r', s')$ for d
 - (Anyone can pretend to be someone else)
 - Ecdsa verification is to verify:
 - $s^{-1}(eG + rP) = (x', y') = R'$, $r' = x' \bmod n == r$?
 - To forge, choose $u, v \in \mathbb{F}_n^*$
 - Compute $R' = (x', y') = uG + vP$
 - Choose $r' = x' \bmod n$, to pass verification, we need
 - $s'^{-1}(e'G + r'P) = uG + vP$
 - $s'^{-1}e' = u \bmod n \rightarrow e' = r'uv^{-1} \bmod n$
 - $s'^{-1}r' = v \bmod n \rightarrow s' = r'v^{-1} \bmod n$
 - $\sigma' = (r', s')$ is a valid signature of e' with secret key d
- *Project: check repo <https://github.com/bitcoin-core/secp256k1>, write a report on the bug fix, performance improvement related to crypto algo usage in bitcoin, figure out the why and the math behind

图 1.1 ECDSA 消息摘要未检查时的签名构造及实验要求

ECDSA (Elliptic Curve Digital Signature Algorithm, 椭圆曲线数字签名算法) 是一种基于椭圆曲线离散对数困难问题构造的数字签名方案，被广泛应用于区块链、数字货币、身份认证和安全通信等场景 [1]。Bitcoin 采用 secp256k1 椭圆曲线，其 ECDSA 签名、验证及相关椭圆曲线运算主要由高性能密码库 libsecp256k1 实现。

在规范的数字签名流程中，签名者首先对原始消息 m 计算密码学摘要：

$$e = H(m),$$

然后使用私钥对摘要 e 进行签名。验证者收到消息和签名后，也应当根据收到的原始消息重新计算：

$$e = H(m),$$

再将摘要、签名和公钥交给 ECDSA 验证算法。验证所使用的摘要必须由收到的原始消息独立计算，即应满足 $e = H(m)$ ；否则，验证成功只能说明摘要、签名和公钥满足数学关系，不能说明签名对应于该原始消息。

然而，底层密码库通常只负责完成密码学运算。以 libsecp256k1 为例，其 ECDSA 验证接口接收的是一个已经计算完成的 32 字节消息摘要，而不是原始消息本身。因此，底层库只能判断给定的摘要、签名和公钥是否满足 ECDSA 验证方程，无法判断该摘要是否确实由某条真实消息计算得到。

若上层程序没有自行对真实消息进行哈希，而是直接接受外部提供的摘要，攻击者便可以利用

ECDSA 验证方程的代数结构，在不知道签名私钥的条件下构造一组相互匹配的：

$$(e', r', s'),$$

使签名 (r', s') 能够在摘要 e' 下通过验证。

不过，这种构造并不表示 ECDSA 的数学安全性被攻破，也不意味着攻击者能够为任意指定的真实消息生成有效签名。攻击者能够控制的是摘要 e' 与签名 (r', s') 的组合，而不能任意指定消息 m 并保证：

$$H(m) = e'.$$

当哈希函数具有良好的原像抗性时，根据给定摘要反向寻找对应消息在计算上不可行。因此，本实验研究的本质是消息摘要未正确绑定所导致的接口误用风险，而不是对 ECDSA 数字签名算法本身的破解。

除接口使用方式外，真实密码库的安全性还受到源码实现、编译器优化、常数时间性质、内存访问模式和底层运算性能等多方面因素影响。即使密码算法的数学设计是安全的，错误的接口调用、秘密相关分支或不合理的底层实现仍可能引入新的安全风险。因此，本实验进一步选择 `libsecp256k1` 中一个常数时间安全修复和一个 ECDSA 验证性能改进进行分析，同时考察其数学定义、接口使用和具体实现。

1.2 实验题目

本实验以 Bitcoin Core 官方 `libsecp256k1` 密码库为实验对象，围绕自选摘要条件下的 ECDSA 签名构造、源码安全分析和性能改进验证三个方面展开，主要完成以下任务：

- (1) 获取并编译 Bitcoin Core 官方 `libsecp256k1` 源码，运行官方测试程序和 benchmark，确认 ECDSA 签名、验证、公钥生成等基本功能能够正常工作；
- (2) 根据图 1.1 中给出的构造思路，在不知道目标私钥的条件下，仅利用目标公钥随机选择 $u, v \in \mathbb{Z}_n^*$ ，计算

$$R' = uG + vP,$$

并进一步构造摘要 e' 和签名 (r', s') ；

- (3) 使用官方 `secp256k1_ecdsa_verify` 接口验证构造得到的 (e', r', s') ，判断其是否能够通过真实 `libsecp256k1` 的 ECDSA 验证；
- (4) 对真实消息 Pay Alice 1 BTC 计算 SHA-256 摘要，并使用同一组构造签名进行验证，完成负面对照实验，说明该构造不能用于任意指定的真实消息；
- (5) 追踪 `libsecp256k1` 中 `secp256k1_ecdsa_verify` 的实现过程，将公开接口、内部验证函数和椭圆曲线点运算与标准 ECDSA 验证公式逐项对应；
- (6) 分析 `libsecp256k1` v0.3.1 中针对 Clang 编译器常数时间问题的安全修复，说明秘密相关控制流、秘密相关内存访问以及 `volatile` 修复方法的作用；
- (7) 分别编译 `libsecp256k1` v0.4.0 和 v0.4.1，在相同实验环境和编译参数下运行官方 benchmark，比较 ECDSA 验证平均耗时，并分析性能改进的具体原因。

2 理论基础

本节首先介绍 secp256k1 曲线和 ECDSA 的签名验证过程，从验证方程出发推导自选摘要签名的构造方法，最后说明该构造的适用条件以及 libsecp256k1 中 lower-S 规范对实验实现的影响。

2.1 secp256k1 与 ECDSA 签名验证原理

secp256k1 是定义在素域 $\mathbb{F}_p$ 上的椭圆曲线，其参数由 SEC 2 标准给出 [2]，曲线方程为：

$$E: y^2 \equiv x^3 + 7 \pmod{p}.$$

曲线上选定基点 G ，其阶为 n ，满足：

$$nG = \mathcal{O},$$

其中， $\mathcal{O}$ 表示无穷远点。由 G 生成的循环子群记为 $\langle G \rangle$ 。ECDSA 中的标量加法、乘法和求逆均按模 n 进行，曲线点坐标运算则按模 p 进行。

签名者的私钥为：

$$d \in \mathbb{Z}_n^*,$$

对应公钥为：

$$P = dG.$$

从已知公钥 P 求出私钥 d ，等价于求解椭圆曲线离散对数问题。ECDSA 的计算安全性建立在该问题难以求解的基础上。

实验中使用的主要符号如表 2.1 所示。

表 2.1 ECDSA 主要符号及含义

符号	含义
p	secp256k1 曲线所在有限域的模数
G	secp256k1 曲线的基点
n	基点 G 的阶
$\mathcal{O}$	椭圆曲线加法群中的无穷远点
d	签名者私钥
$P = dG$	签名者公钥
m	原始消息
e	由 $H(m)$ 转换得到的消息摘要标量
k	每次签名使用的临时标量
(r, s)	ECDSA 签名

给定私钥 d 和消息 m ，签名者首先计算消息摘要标量：

$$e = H(m).$$

随后选择临时标量:

$$k \in \mathbb{Z}_n^*,$$

并计算曲线点:

$$R = kG = (x_R, y_R).$$

签名的两个分量分别为:

$$r = x_R \bmod n,$$

$$s = k^{-1}(e + dr) \bmod n.$$

若 $r = 0$ 或 $s = 0$, 则需要重新选择 k 。最终签名为:

$$\sigma = (r, s).$$

临时标量 k 必须保密, 且不能在不同消息之间重复使用。若两个不同摘要 e_1 和 e_2 使用相同的 k 生成签名 (r, s_1) 和 (r, s_2) , 则有:

$$k = (e_1 - e_2)(s_1 - s_2)^{-1} \pmod{n},$$

进而可以恢复私钥:

$$d = (s_1 k - e_1)r^{-1} \pmod{n}.$$

实际实现可以按照 RFC 6979, 从私钥和消息摘要确定性地生成临时标量, 以降低随机数生成不当或重复使用 k 的风险 [3]。

验证者已知公钥 P 、摘要 e 和签名 (r, s) 。首先检查:

$$1 \leq r < n, \quad 1 \leq s < n.$$

随后计算:

$$w = s^{-1} \bmod n,$$

$$u_1 = ew \bmod n, \quad u_2 = rw \bmod n,$$

并得到验证点:

$$R_v = u_1G + u_2P.$$

若 $R_v = \mathcal{O}$, 则验证失败; 否则检查:

$$x(R_v) \bmod n \stackrel{?}{=} r.$$

对于合法签名，有：

$$\begin{aligned} R_v &= s^{-1}eG + s^{-1}rP \\ &= s^{-1}(e + rd)G \\ &= k(e + rd)^{-1}(e + rd)G \\ &= kG \\ &= R. \end{aligned}$$

因此：

$$x(R_v) \bmod n = r,$$

验证通过。

2.2 自选摘要签名构造及正确性分析

设目标公钥为：

$$P = dG.$$

攻击者知道公钥 P ，但不知道私钥 d 。构造过程中同时选择摘要和签名，而不是为预先给定的消息生成签名。

首先选择两个非零标量：

$$u, v \in \mathbb{Z}_n^*,$$

并计算：

$$R_f = uG + vP = (x_f, y_f).$$

令：

$$r' = x_f \bmod n.$$

若 $r' = 0$ ，则重新选择 u 和 v 。

为了使验证算法内部计算出的两个标量分别等于 u 和 v ，要求：

$$s'^{-1}e' = u,$$

以及：

$$s'^{-1}r' = v.$$

由第二个等式可以得到：

$$s' = r'v^{-1} \bmod n.$$

再由第一个等式得到：

$$e' = us' \bmod n,$$

即：

$$e' = ur'v^{-1} \bmod n.$$

验证者收到摘要 e' 、签名 (r', s') 和公钥 P 后，会计算：

$$u'_1 = s'^{-1} e' = u,$$

以及：

$$u'_2 = s'^{-1} r' = v.$$

因此，验证点为：

$$\begin{aligned} R'_v &= u'_1 G + u'_2 P \\ &= uG + vP \\ &= R_f. \end{aligned}$$

由于：

$$r' = x(R_f) \bmod n,$$

所以：

$$x(R'_v) \bmod n = r',$$

签名 (r', s') 能够在摘要 e' 下通过 ECDSA 验证。

该构造只使用了公钥 P 、基点 G 以及攻击者选择的 u 和 v ，计算过程中不需要私钥 d 。

2.3 构造的适用范围与 lower-S 处理

上述方法同时构造摘要 e' 和签名 (r', s') 。它不能直接用于为预先指定的消息 m_0 生成签名，因为此时消息摘要已经固定为：

$$e_0 = H(m_0).$$

一般情况下：

$$e_0 \neq e'.$$

若要使构造签名适用于消息 m_0 ，必须满足：

$$H(m_0) = e'.$$

也可以尝试寻找另一条消息 m ，使：

$$H(m) = e'.$$

当哈希函数具有原像抗性时，从随机给定的摘要 e' 找到对应消息在计算上不可行。因此，该构造不能为任意指定消息生成有效签名，其成立依赖于验证者直接接受外部提供的摘要。

此外，libsecp256k1 的 ECDSA 验证接口只接受 lower-S 形式的签名，即：

$$s \leq \frac{n}{2}.$$

若按照构造公式得到：

$$s' > \frac{n}{2},$$

则将其规范化为:

$$\tilde{s} = n - s'.$$

由于:

$$\tilde{s} \equiv -s' \pmod{n},$$

所以:

$$\tilde{s}^{-1} \equiv -s'^{-1} \pmod{n}.$$

此时验证算法计算出的两个系数变为:

$$\tilde{s}^{-1}e' = -u \pmod{n},$$

$$\tilde{s}^{-1}r' = -v \pmod{n}.$$

对应的验证点为:

$$\begin{aligned}\tilde{R}_v &= (-u)G + (-v)P \\ &= -(uG + vP) \\ &= -R_f.\end{aligned}$$

若:

$$R_f = (x_f, y_f),$$

则:

$$-R_f = (x_f, -y_f \bmod p).$$

两个点的横坐标相同, 因此:

$$x(-R_f) \bmod n = x(R_f) \bmod n = r'.$$

所以, 将 s' 替换为 $n - s'$ 不会改变 ECDSA 的验证结果, 同时满足 libsecp256k1 对 lower-S 签名的要求。

3 实验环境与程序设计

本实验包括 ECDSA 构造验证、官方源码分析、安全修复分析和版本性能对比四部分。为保证各部分结果具有可比性, 实验均在同一 Ubuntu 虚拟机中完成, 并尽量保持编译器、优化参数和运行条件一致。

3.1 实验环境与版本安排

本实验使用 C 语言编写构造程序, 调用 Bitcoin Core 官方 libsecp256k1 [5] 完成椭圆曲线点运算、ECDSA 签名和验证, 使用 OpenSSL 提供的大整数运算、随机数生成和 SHA-256 接口完成辅助计算。实验环境如表 3.1 所示。

表 3.1 实验环境与软件配置

项目	配置
操作系统	Ubuntu 22.04 虚拟机
处理器架构	x86-64
编程语言	C
编译器	GCC
性能实验编译优化参数	-O2
主实验密码库	Bitcoin Core libsecp256k1 v0.7.1
性能对比版本	libsecp256k1 v0.4.0 与 v0.4.1
辅助密码库	OpenSSL libcrypto
OpenSSL 功能	BIGNUM 模运算、随机标量生成、SHA-256 哈希
主实验构建方式	CMake
性能实验构建方式	Autotools 与 Make
结果记录方式	Shell 脚本、tee 和文本文件

主实验采用 libsecp256k1 v0.7.1。该版本用于编译官方库、运行官方 benchmark，并调用公开接口完成 ECDSA 构造验证。性能实验则分别检出 v0.4.0 和 v0.4.1，用于比较版本更新前后的 ECDSA 验证耗时。

两个性能对比版本通过 git worktree 放置在独立目录中：

```
secp-v040    -> libsecp256k1 v0.4.0
secp-v041    -> libsecp256k1 v0.4.1
```

采用独立工作目录可以避免不同版本之间共享配置缓存、目标文件和可执行程序，从而减少旧编译产物对测试结果的影响。

实验程序通过动态链接方式调用已编译的 libsecp256k1。OpenSSL 负责模逆、模乘、随机数和 SHA-256 等辅助运算；椭圆曲线点生成、点乘、点加和最终签名验证均由官方 libsecp256k1 完成，避免自行实现曲线运算带来的偏差。

3.2 总体流程与控制变量

本实验按照先验证基础环境、再完成构造实验、随后分析源码与安全修复、最后开展性能对比的顺序进行。各部分所使用的代码和数据相互独立，但后续分析均建立在官方密码库能够正常编译和运行的基础上。

首先，使用 libsecp256k1 v0.7.1 完成官方库编译和基准测试，确认 ECDSA 签名、验证及椭圆曲线基本运算能够正常执行。在此基础上，编写 forge_demo.c，分别完成正常签名验证、自选摘要签名构造和真实消息负面对照。

构造实验完成后，进一步查看官方验证接口及其内部实现，将程序中的验证结果与 ECDSA 数学公式对应。随后选取 v0.3.1 中的常数时间修复进行分析，说明编译器优化可能引入的侧信道风险。最后，在相同编译条件下分别测试 v0.4.0 和 v0.4.1，比较两个版本的 ECDSA 验证耗时。

各阶段的主要内容和预期结果如表 3.2 所示。

表 3.2 实验阶段及主要内容

阶段	主要内容	预期产出
官方库准备	编译 v0.7.1, 运行官方 benchmark	确认签名、验证、公钥生成等功能能够正常运行
构造实验	编写 <code>forge_demo.c</code> , 构造 (e', r', s')	获得正常签名、自选摘要构造和真实消息对照结果
源码分析	追踪公开验证接口、内部验证函数和横坐标检查	将官方实现与 ECDSA 验证公式对应
安全修复分析	查看 v0.3.1 的常数时间修复及对应提交	说明编译器优化产生的风险及修改方法
性能实验	对比 v0.4.0 和 v0.4.1 的官方 benchmark	计算 ECDSA 验证平均耗时及下降比例

性能对比中保持运行环境、编译器、优化参数、benchmark 程序、预热次数和正式测试次数一致。具体配置、测试命令及计算方法在第 6 节中说明, 避免在本章重复展开。

3.3 程序设计与判定方法

实验核心程序为:

`forge_demo.c`

程序包含正常签名验证、自选摘要签名构造和真实消息负面对照三个部分, 其输入、处理过程和判定方法如表 3.3 所示。

表 3.3 构造程序的主要模块

模块	主要处理	判定方法
正常签名验证	生成私钥和公钥, 对真实消息计算 SHA-256, 并使用私钥签名	官方验证接口返回 1, 程序输出 PASS
自选摘要构造	仅使用公钥计算 $uG + vP$, 构造 r' 、 s' 和 e'	构造签名在摘要 e' 下通过官方验证
真实消息对照	保持构造签名不变, 将摘要替换为真实消息的 SHA-256	官方验证接口返回 0, 程序输出预期的 FAIL

程序首先生成一个随机私钥 d , 并调用官方接口计算公钥:

$$P = dG.$$

该私钥仅用于生成实验中的目标公钥和正常签名, 不会传入自选摘要构造函数。核心构造函数的接口为:

```
1 static int forge_chosen_hash_signature(  
2     const secp256k1_context *ctx,  
3     const secp256k1_pubkey *victim_pubkey,  
4     unsigned char chosen_hash[32],  
5     secp256k1_ecdsa_signature *forged_signature
```

```
6 );
```

从参数可以看出，函数只接收目标公钥 `victim_pubkey`，不接收目标私钥。函数内部生成 u 和 v ，调用官方公钥接口计算 uG 、 vP 和 $R' = uG + vP$ ，随后使用 OpenSSL BIGNUM 计算 r' 、 s' 与 e' ，并对 s' 进行 lower-S 处理。具体 API 调用和关键代码在第 4.2 节中给出。

程序每次运行都会重新生成目标私钥、 u 和 v ，因此输出的摘要 e' 、签名分量 r' 和 s' 会发生变化。实验以三项验证结果作为判定依据：正常签名应通过，自选摘要构造应通过，真实消息负面对照应失败。

4 ECDSA 自选摘要签名构造实验

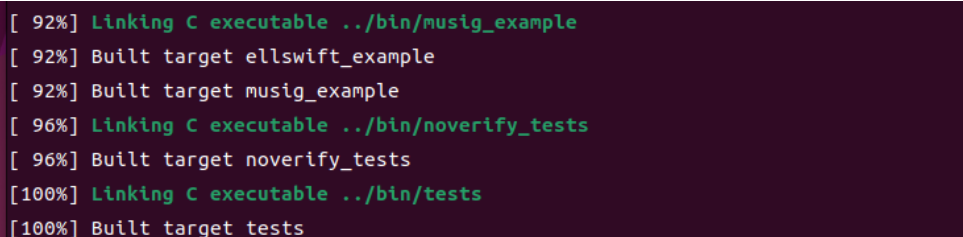
本节在 `libsecp256k1 v0.7.1` 上完成自选摘要签名构造。实验首先编译并测试官方库，确认其基本功能正常；随后编写 `forge_demo.c`，分别完成正常签名验证、自选摘要签名构造和真实消息负面对照。

4.1 官方库编译与基线验证

本实验使用 Bitcoin Core 官方发布的 `libsecp256k1 v0.7.1`。完成 CMake 配置后，使用以下命令编译密码库、测试程序、示例程序和 benchmark：

```
1 cmake --build build -j"${nproc}"
```

编译输出如图 4.1 所示。构建过程完成至 100%，测试程序 `tests`、`noverify_tests` 以及相关示例程序均成功生成，未出现链接错误。



```
[ 92%] Linking C executable ../bin/musig_example
[ 92%] Built target ellswift_example
[ 92%] Built target musig_example
[ 96%] Linking C executable ../bin/noverify_tests
[ 96%] Built target noverify_tests
[100%] Linking C executable ../bin/tests
[100%] Built target tests
```

图 4.1 `libsecp256k1 v0.7.1` 编译完成

编译成功只能说明源码能够生成可执行程序，不能直接证明密码功能运行正确。因此，继续调用 CTest 执行官方测试：

```
1 ctest --test-dir build --output-on-failure
```

测试结果如图 4.2 所示。包括普通测试、穷举测试、ECDSA 示例、ECDH 示例、Schnorr 签名示例、EllSwift 示例和 MuSig 示例在内的 8 项测试均通过，未出现失败用例。

```
openhitals@1111:~/桌面/secp256k1$ ctest --test-dir build --output-on-failure
Internal ctest changing into directory: /home/openhitals/桌面/secp256k1/build
Test project /home/openhitals/桌面/secp256k1/build
  Start 1: secp256k1_noverify_tests
1/8 Test #1: secp256k1_noverify_tests ..... Passed    12.87 sec
  Start 2: secp256k1_tests
2/8 Test #2: secp256k1_tests ..... Passed    28.44 sec
  Start 3: secp256k1_exhaustive_tests
3/8 Test #3: secp256k1_exhaustive_tests ..... Passed     5.49 sec
  Start 4: secp256k1_ecdsa_example
4/8 Test #4: secp256k1_ecdsa_example ..... Passed     0.00 sec
  Start 5: secp256k1_ecdh_example
5/8 Test #5: secp256k1_ecdh_example ..... Passed     0.00 sec
  Start 6: secp256k1_schnorr_example
6/8 Test #6: secp256k1_schnorr_example ..... Passed     0.00 sec
  Start 7: secp256k1_ellswift_example
7/8 Test #7: secp256k1_ellswift_example ..... Passed     0.00 sec
  Start 8: secp256k1_musig_example
8/8 Test #8: secp256k1_musig_example ..... Passed     0.00 sec

100% tests passed, 0 tests failed out of 8

Total Test time (real) = 46.81 sec
```

图 4.2 libsecp256k1 官方测试结果

测试结果中的：

```
100% tests passed, 0 tests failed out of 8
```

表明当前编译得到的库能够正确完成测试集覆盖的密码运算。此次测试总耗时为 46.81 秒，其中普通测试和无验证模式测试耗时较长，其余示例程序均在较短时间内完成。

在功能测试通过后，运行官方 benchmark：

```
1 ./build/bin/bench
```

benchmark 分别统计各项密码运算的最小耗时、平均耗时和最大耗时，单位为微秒。运行结果如图 4.3 所示。

```
openhitals@1111:~/桌面/secp256k1$ ./build/bin/bench
```

Benchmark	Min(us)	Avg(us)	Max(us)
ecdsa_verify	25.6	25.8	26.3
ecdsa_sign	17.1	17.7	20.0
ec_keygen	11.6	11.7	12.0
ecdh	24.7	24.8	24.9
schnorrsg_sign	12.3	12.5	12.7
schnorrsg_verify	26.0	26.2	26.5
ellswift_encode	14.9	15.1	15.5
ellswift_decode	6.41	6.48	6.57
ellswift_keygen	26.7	26.9	27.4
ellswift_ecdh	27.6	27.9	28.2

图 4.3 libsecp256k1 v0.7.1 官方基准测试结果

与本实验关系较密切的测试数据整理如表 4.1 所示。

表 4.1 libsecp256k1 v0.7.1 部分基准测试结果

测试项目	Min/ μs	Avg/ μs	Max/ μs
ecdsa_verify	25.6	25.8	26.3
ecdsa_sign	17.1	17.7	20.0
ec_keygen	11.6	11.7	12.0
ecdh	24.7	24.8	24.9

在当前虚拟机环境中，一次 ECDSA 签名的平均耗时为 $17.7 \mu s$ ，一次 ECDSA 验证的平均耗时为 $25.8 \mu s$ 。官方测试和 benchmark 均能正常运行，说明后续构造实验所调用的公钥生成、点运算、签名解析和验证接口处于可用状态。

4.2 构造程序设计与实现

实验程序 `forge_demo.c` 包含三部分：

- (1) 使用随机私钥完成一次正常 ECDSA 签名和验证；
- (2) 仅根据目标公钥构造摘要 e' 和签名 (r', s') ；
- (3) 保持构造签名不变，将摘要替换为真实消息的 SHA-256，检查验证结果。

程序首先创建 `secp256k1` 上下文，随机生成合法私钥，并计算目标公钥：

```
1 secp256k1_context *ctx =
2     secp256k1_context_create(SECP256K1_CONTEXT_NONE);
3
4 random_valid_scalar(ctx, victim_seckey);
5
6 secp256k1_ec_pubkey_create(
7     ctx,
```

```
8     &victim_pubkey,  
9     victim_seckey  
10 );
```

这里生成的私钥仅用于建立实验中的目标公钥，并完成正常签名对照。该私钥不会传入自选摘要构造函数。

正常签名部分首先对消息进行 SHA-256 计算：

```
1  const unsigned char real_message[] =  
2      "Pay Alice 1 BTC";  
3  
4  SHA256(  
5      real_message,  
6      strlen((const char *)real_message),  
7      real_message_hash  
8  );
```

随后调用官方签名和验证接口：

```
1  secp256k1_ecdsa_sign(  
2      ctx,  
3      &normal_signature,  
4      real_message_hash,  
5      victim_seckey,  
6      NULL,  
7      NULL  
8  );  
9  
10 normal_result = secp256k1_ecdsa_verify(  
11     ctx,  
12     &normal_signature,  
13     real_message_hash,  
14     &victim_pubkey  
15 );
```

这一部分用于确认私钥、公钥、消息摘要、签名对象和官方验证接口之间能够正常配合。若该步骤失败，则无法判断后续构造失败是由数学关系还是实验环境引起的。

自选摘要构造函数的接口如下：

```
1  static int forge_chosen_hash_signature(  
2      const secp256k1_context *ctx,  
3      const secp256k1_pubkey *victim_pubkey,  
4      unsigned char chosen_hash[32],  
5      secp256k1_ecdsa_signature *forged_signature  
6  );
```

函数参数中只有目标公钥 `victim_pubkey`，没有目标私钥 `victim_seckey`。因此，构造过程无法直接读取或使用目标私钥。

程序随机生成两个合法非零标量 u 和 v ，然后调用官方接口分别计算 uG 和 vP ：

```
1  secp256k1_ec_pubkey_create(  
2      ctx,
```

```
3     &point_uG,  
4     u32  
5 );  
6  
7 point_vP = *victim_pubkey;  
8  
9 secp256k1_ec_pubkey_tweak_mul(  
10    ctx,  
11    &point_vP,  
12    v32  
13 );
```

其中, `point_uG` 表示点 uG , `point_vP` 表示点 vP 。随后使用官方公钥组合接口完成点加运算:

```
1 const secp256k1_pubkey *points_to_combine[2];  
2  
3 points_to_combine[0] = &point_uG;  
4 points_to_combine[1] = &point_vP;  
5  
6 secp256k1_ec_pubkey_combine(  
7    ctx,  
8    &point_R,  
9    points_to_combine,  
10    2  
11 );
```

由此得到:

$$R' = uG + vP.$$

程序将 `point_R` 序列化为未压缩公钥格式:

$$04\|x'\|y',$$

其中第一个字节 04 为未压缩公钥前缀, 随后 32 字节为横坐标 x' , 最后 32 字节为纵坐标 y' 。提取横坐标后, 将其按模 n 归约, 得到:

$$r' = x' \bmod n.$$

相应代码为:

```
1 secp256k1_ec_pubkey_serialize(  
2    ctx,  
3    serialized_point,  
4    &serialized_length,  
5    &point_R,  
6    SECP256K1_EC_UNCOMPRESSED  
7 );  
8  
9 BN_bin2bn(  
10    serialized_point + 1,  
11    32,  
12    r_bn
```



```
13 );  
14  
15 BN_mod(  
16     r_bn,  
17     r_bn,  
18     order,  
19     bn_ctx  
20 );
```

接下来使用 OpenSSL BIGNUM 计算构造所需的标量：

```
1 BN_mod_inverse(  
2     v_inverse,  
3     v_bn,  
4     order,  
5     bn_ctx  
6 );  
7  
8 BN_mod_mul(  
9     s_bn,  
10    r_bn,  
11    v_inverse,  
12    order,  
13    bn_ctx  
14 );  
15  
16 BN_mod_mul(  
17     e_bn,  
18     u_bn,  
19     s_bn,  
20     order,  
21     bn_ctx  
22 );
```

三步分别对应：

$$v^{-1} \bmod n,$$
$$s' = r'v^{-1} \bmod n,$$

以及：

$$e' = us' \bmod n.$$

由于官方验证接口仅接受 lower-S 签名，程序还需要检查 s' 是否大于 $n/2$ ：

```
1 if (BN_cmp(s_bn, half_order) > 0) {  
2     BN_sub(  
3         s_bn,  
4         order,  
5         s_bn  
6     );  
7 }
```

若条件成立，则执行：

$$s' \leftarrow n - s'.$$

随后将 r' 和 s' 转换为两个固定长度的 32 字节整数，并拼接为 64 字节 compact signature：

```
1 bn_to_bytes32(  
2     r_bn,  
3     compact_signature  
4 );  
5  
6 bn_to_bytes32(  
7     s_bn,  
8     compact_signature + 32  
9 );  
10  
11 secp256k1_ecdsa_signature_parse_compact(  
12     ctx,  
13     forged_signature,  
14     compact_signature  
15 );
```

构造完成后，调用官方接口验证摘要 e' 和签名 (r', s') ：

```
1 forged_result = secp256k1_ecdsa_verify(  
2     ctx,  
3     &forged_signature,  
4     chosen_hash,  
5     &victim_pubkey  
6 );
```

负面对照不修改构造签名，只将输入摘要替换为真实消息的 SHA-256：

```
1 negative_control_result =  
2     secp256k1_ecdsa_verify(  
3         ctx,  
4         &forged_signature,  
5         real_message_hash,  
6         &victim_pubkey  
7     );
```

这样可以直接比较同一签名在构造摘要和真实消息摘要下的验证结果。

4.3 实验结果与分析

实验运行结果如图 4.4 所示。

```
[1] Normal ECDSA signature
Normal signature verification: PASS

[2] Chosen-hash signature construction
The forging function receives only the public key, not the private key.
Chosen digest e' : 20bbc31f75be9b24c22e67f8faaca125efecce5ce0366530b0604c6e74e9941c
Forged r'       : 8597120da136e5ba1525d8de31d9b0c7f2ad178ac5fdfe5b0bd23947e1fa0b31
Forged s'       : 1cddec1e828afef77c35b0d03a170e1dc24d4ebb01170ea829f49fa76a34aae9
Verification on chosen digest: PASS

[3] Negative control using a real message
Message: Pay Alice 1 BTC
SHA256(message) : 44b8841b753dd6b72b90dfb80f64fedcb869d00d0c24dd68a8f8ad0cc96b8643
Same signature on SHA256(message): FAIL (expected)
```

图 4.4 ECDSA 自选摘要签名构造及负面对照结果

本次运行中，程序输出了构造摘要 e' 、签名分量 r' 和 s' 。由于目标私钥、 u 和 v 均由随机数生成器产生，因此这些十六进制数值会随每次运行而改变。实验判断不依赖某一组固定数值，而取决于三项验证结果是否满足预期关系。

结果汇总如表 4.2 所示。

表 4.2 ECDSA 自选摘要签名构造实验结果

实验项目	实际结果	结果说明
正常 ECDSA 签名验证	PASS	目标密钥生成、消息哈希、签名和验证接口工作正常
自选摘要签名构造	PASS	构造得到的摘要 e' 和签名 (r', s') 满足官方验证条件
真实消息负面对照	FAIL (符合预期)	构造签名不能直接用于 Pay Alice 1 BTC 的 SHA-256 摘要

第一组结果为：

```
Normal signature verification: PASS
```

说明官方库能够正确验证由目标私钥正常生成的签名，也说明后续实验使用的目标公钥和运行环境没有异常。

第二组结果为：

```
Verification on chosen digest: PASS
```

构造函数只接收目标公钥，却能够生成通过官方验证的摘要和签名。若构造得到的 s' 已满足 lower-S 要求，官方验证函数内部计算出的两个系数分别为 u 和 v ，验证点为 $R' = uG + vP$ 。若程序将高 s' 规范化为 $n - s'$ ，两个系数同时变为 $-u$ 和 $-v$ ，验证点变为 $-R'$ 。由于 R' 与 $-R'$ 的横

坐标相同，两种情况下均能通过最终的横坐标检查。本次截图中的 s' 已处于 lower-S 范围，因此没有触发规范化。

第三组结果为：

```
Same signature on SHA256(message): FAIL (expected)
```

此时签名 (r', s') 没有变化，但验证摘要由构造的 e' 替换为：

$$H(\text{Pay Alice 1 BTC}).$$

新的摘要通常不会使验证系数保持为构造阶段对应的 u （或 lower-S 规范化后的 $-u$ ），因此验证函数重新计算的点不再是 R' 或 $-R'$ ，横坐标检查随之失败。

三组结果共同说明，本实验构造的是一组彼此匹配的摘要和签名，而不是对预先指定消息的签名。若验证程序从原始消息中自行计算摘要，构造签名无法通过验证；只有在验证程序直接接受外部提供的摘要时，该构造才具备实际影响。

5 libsecp256k1 验证源码与安全修复分析

本节结合 libsecp256k1 v0.7.1 的实际源码，分析公开 ECDSA 验证接口、内部验证计算和横坐标判定方法，并进一步研究 v0.3.1 中与常数时间实现有关的安全修复。

5.1 ECDSA 验证接口与内部计算

libsecp256k1 的公开接口 `secp256k1_ecdsa_verify` 接收签名对象、32 字节消息摘要和公钥。接口参数中不包含原始消息，因此该函数无法自行判断传入的摘要是否确实由某条消息计算得到。

公开头文件对 `msghash32` 的使用作出了明确要求：验证者应当自行对收到的消息使用密码学哈希函数，不应直接接受外部提供的 32 字节摘要。若上层程序忽略这一要求，底层验证函数仍会按照 ECDSA 方程处理该摘要，而不会检查摘要与原始消息之间的关系。

头文件中的说明如下：

```
1 /* The verifier must make sure to apply a cryptographic
2  * hash function to the message by itself and not accept an
3  * msghash32 value directly. Otherwise, it would be easy to
4  * create a valid signature without knowledge of the secret key.
5  */
```

公开验证函数的实现如图 5.1 所示。

```
458 int secp256k1_ecdsa_verify(const secp256k1_context* ctx, const secp256k1_ecdsa_signature *sig, const unsigned char *msghash32, const secp256k1_pubkey *pubkey) {
459     secp256k1_ge q;
460     secp256k1_scalar r, s;
461     secp256k1_scalar m;
462     VERIFY_CHECK(ctx != NULL);
463     ARG_CHECK(msghash32 != NULL);
464     ARG_CHECK(sig != NULL);
465     ARG_CHECK(pubkey != NULL);
466
467     secp256k1_scalar_set_b32(&m, msghash32, NULL);
468     secp256k1_ecdsa_signature_load(ctx, &r, &s, sig);
469     return (!secp256k1_scalar_is_high(&s) &&
470             secp256k1_pubkey_load(ctx, &q, pubkey) &&
471             secp256k1_ecdsa_sig_verify(&r, &s, &q, &m));
472 }
```

图 5.1 secp256k1_ecdsa_verify 公开验证接口

函数首先执行参数检查，确认上下文、消息摘要、签名和公钥均不为空。随后将外部传入的 32 字节摘要转换为内部标量：

```
1 secp256k1_scalar_set_b32(
2     &m,
3     msghash32,
4     NULL
5 );
```

变量 m 表示消息摘要对应的标量，而不是原始消息。函数内部没有调用 SHA-256，也没有接收用于重新计算摘要的消息数据。

随后，程序从签名对象中读取签名分量 r 和 s ：

```
1 secp256k1_ecdsa_signature_load(
2     ctx,
3     &r,
4     &s,
5     sig
6 );
```

返回语句依次完成三个检查：

```
1 return (
2     !secp256k1_scalar_is_high(&s) &&
3     secp256k1_pubkey_load(ctx, &q, pubkey) &&
4     secp256k1_ecdsa_sig_verify(&r, &s, &q, &m)
5 );
```

其中：

- (1) secp256k1_scalar_is_high 检查签名是否满足 lower-S 规范；
- (2) secp256k1_pubkey_load 将公开公钥对象转换为内部椭圆曲线点；
- (3) secp256k1_ecdsa_sig_verify 执行具体的 ECDSA 数学验证。

具体的验证运算位于 `src/ecdsa_impl.h` 中，其关键代码如图 5.2 所示。

```
195 static int secp256k1_ecdsa_sig_verify(const secp256k1_scalar *sigr, const secp256k1_scalar *sigs, const secp256k1_ge *pubkey, const secp256k1_scalar *message) {
196     unsigned char c[32];
197     secp256k1_scalar sn, u1, u2;
198     #if !defined(EXHAUSTIVE_TEST_ORDER)
199         secp256k1_fe xr;
200     #endif
201     secp256k1_gej pubkeyj;
202     secp256k1_gej pr;
203
204     if (secp256k1_scalar_is_zero(sigr) || secp256k1_scalar_is_zero(sigs)) {
205         return 0;
206     }
207
208     secp256k1_scalar_inverse_var(&sn, sigs);
209     secp256k1_scalar_mul(&u1, &sn, message);
210     secp256k1_scalar_mul(&u2, &sn, sigr);
211     secp256k1_gej_set_ge(&pubkeyj, pubkey);
212     secp256k1_ecmult(&pr, &pubkeyj, &u2, &u1);
213     if (secp256k1_gej_is_infinity(&pr)) {
214         return 0;
215     }
```

图 5.2 ECDSA 内部验证函数的核心计算

内部函数首先检查：

$$r \neq 0, \quad s \neq 0.$$

若任一签名分量为零，函数立即返回失败。非零检查通过后，程序计算 s 在模 n 意义下的逆元：

```
1 secp256k1_scalar_inverse_var(
2     &sn,
3     sigs
4 );
```

因此：

$$sn = s^{-1} \bmod n.$$

这里使用可变时间的 `secp256k1_scalar_inverse_var`，是因为验证阶段的摘要、签名和公钥均为公开输入，不涉及需要保密的私钥标量。与私钥相关的签名生成运算仍需采用常数时间实现。

随后计算：

```
1 secp256k1_scalar_mul(
2     &u1,
3     &sn,
4     message
5 );
6
7 secp256k1_scalar_mul(
8     &u2,
```

```

9     &sn,
10    sigr
11 );

```

对应的数学表达式为：

$$u_1 = s^{-1}e \bmod n,$$

$$u_2 = s^{-1}r \bmod n.$$

公钥由仿射坐标转换为 Jacobian 坐标后，程序调用：

```

1 secp256k1_ecmult(
2     &pr,
3     &pubkeyj,
4     &u2,
5     &u1
6 );

```

该函数计算：

$$pr = u_2P + u_1G.$$

需要注意参数顺序：pubkeyj 对应公钥点 P ，u2 是公钥点的系数，u1 是基点 G 的系数。因此，实际计算结果与标准 ECDSA 验证公式一致：

$$pr = s^{-1}rP + s^{-1}eG.$$

源码变量与数学含义的对应关系如表 5.1 所示。

表 5.1 验证源码变量与数学符号的对应关系

源码变量	数学含义	在本实验构造中的取值
message	摘要 e	$e' = us'$
sigr	签名分量 r	$r' = x(R') \bmod n$
sigs	签名分量 s	$s' = r'v^{-1}$
sn	s^{-1}	s'^{-1}
u1	$s^{-1}e$	u ；若执行 lower-S 规范化则为 $-u$
u2	$s^{-1}r$	v ；若执行 lower-S 规范化则为 $-v$
pr	$u_2P + u_1G$	R' ；若执行 lower-S 规范化则为 $-R'$

若构造出的 s' 已满足 lower-S 要求，则内部计算得到 $u_1 = u$ 、 $u_2 = v$ ，验证点为 R' 。若程序将 s' 规范化为 $n - s'$ ，则两个系数同时变为 $-u$ 和 $-v$ ，验证点为 $-R'$ 。构造程序预先令 $r' = x(R') \bmod n$ ，而 R' 与 $-R'$ 的横坐标相同，因此两种情况下均可通过后续横坐标检查。

5.2 Jacobian 坐标下的横坐标判定

ECDSA 验证需要检查：

$$x(pr) \bmod n = r.$$

若直接将 Jacobian 坐标转换为仿射坐标，需要计算有限域逆元。libsecp256k1 使用等价的坐

标比较方法，避免执行该求逆运算。源码中的推导注释如图 5.3 所示。

```
229     secp256k1_scalar_get_b32(c, sigr);
230     /* we can ignore the fe_set_b32_limit return value, because we know the input is in range */
231     (void)secp256k1_fe_set_b32_limit(&xr, c);
232
233     /** We now have the recomputed R point in pr, and its claimed x coordinate (modulo n)
234     *   in xr. Naively, we would extract the x coordinate from pr (requiring a inversion modulo p),
235     *   compute the remainder modulo n, and compare it to xr. However:
236     *
237     *       xr == X(pr) mod n
238     *   <=> exists h. (xr + h * n < p && xr + h * n == X(pr))
239     *   [Since 2 * n > p, h can only be 0 or 1]
240     *   <=> (xr == X(pr)) || (xr + n < p && xr + n == X(pr))
241     *   [In Jacobian coordinates, X(pr) is pr.x / pr.z^2 mod p]
242     *   <=> (xr == pr.x / pr.z^2 mod p) || (xr + n < p && xr + n == pr.x / pr.z^2 mod p)
243     *   [Multiplying both sides of the equations by pr.z^2 mod p]
244     *   <=> (xr * pr.z^2 mod p == pr.x) || (xr + n < p && (xr + n) * pr.z^2 mod p == pr.x)
245     *
246     *   Thus, we can avoid the inversion, but we have to check both cases separately.
247     *   secp256k1_gej_eq_x implements the (xr * pr.z^2 mod p == pr.x) test.
248     */
```

图 5.3 Jacobian 坐标下横坐标比较的源码推导

设重新计算得到的验证点为：

$$pr = (X : Y : Z).$$

其仿射横坐标为：

$$x(pr) = XZ^{-2} \pmod{p}.$$

朴素方法需要先计算 Z^{-1} ，再求出 XZ^{-2} 。有限域求逆的开销通常高于乘法和平方，因此源码将：

$$x_r = \frac{X}{Z^2} \pmod{p}$$

改写为：

$$x_r Z^2 = X \pmod{p}.$$

这样只需计算有限域平方和乘法，无需显式求出 Z^{-1} 。

ECDSA 签名中的 r 是横坐标模群阶 n 的结果，而点坐标属于有限域 $\mathbb{F}_p$ 。因此，已知：

$$r = x(pr) \bmod n,$$

并不能直接认定：

$$r = x(pr).$$

由于 secp256k1 参数满足：

$$2n > p,$$

而横坐标满足：

$$0 \leq x(pr) < p,$$

所以可能的横坐标只有两种：

$$x(pr) = r,$$

或者:

$$x(pr) = r + n, \quad r + n < p.$$

相应的源码判断如图 5.4 所示。

```
249     if (secp256k1_gej_eq_x_var(&xr, &pr)) {
250         /* xr * pr.z^2 mod p == pr.x, so the signature is valid. */
251         return 1;
252     }
253     if (secp256k1_fe_cmp_var(&xr, &secp256k1_ecdsa_const_p_minus_order) >= 0) {
254         /* xr + n >= p, so we can skip testing the second case. */
255         return 0;
256     }
257     secp256k1_fe_add(&xr, &secp256k1_ecdsa_const_order_as_fe);
258     if (secp256k1_gej_eq_x_var(&xr, &pr)) {
259         /* (xr + n) * pr.z^2 mod p == pr.x, so the signature is valid. */
260         return 1;
261     }
262     return 0;
```

图 5.4 ECDSA 横坐标两种可能情况的源码检查

函数首先执行:

```
1 if (secp256k1_gej_eq_x_var(&xr, &pr)) {
2     return 1;
3 }
```

该函数检查:

$$rZ^2 \equiv X \pmod{p}.$$

若等式成立, 则:

$$x(pr) = r,$$

签名有效。

若第一种情况不成立, 程序检查 $r + n$ 是否仍小于 p 。源码使用:

```
1 if (
2     secp256k1_fe_cmp_var(
3         &xr,
4         &secp256k1_ecdsa_const_p_minus_order
5     ) >= 0
6 ) {
7     return 0;
8 }
```

常量:

$$\text{secp256k1_ecdsa_const_p_minus_order} = p - n.$$

如果:

$$r \geq p - n,$$

则有：

$$r + n \geq p,$$

第二种情况不可能成立，函数直接返回失败。

否则，程序将 n 加到 r 上：

```
1 secp256k1_fe_add(
2     &xr,
3     &secp256k1_ecdsa_const_order_as_fe
4 );
```

再检查：

$$(r + n)Z^2 \equiv X \pmod{p}.$$

若成立，则：

$$x(pr) = r + n,$$

其模 n 的结果仍为 r ，所以签名同样有效。

该实现没有改变 ECDSA 的验证条件，只是避免了 Jacobian 坐标转仿射坐标时的有限域求逆。

5.3 常数时间问题与修复

密码算法在处理私钥、临时标量和中间秘密状态时，需要避免秘密数据影响程序的控制流、内存访问位置或执行时间。时序差异可能被用于推断密码运算中的秘密信息 [4]。

libsecp256k1 v0.3.1 的更新记录中列出了一项与 Clang 编译器有关的常数时间修复，如图 5.5 所示。

```
openhitals@1111:~/桌面/secp256k1$ cd ~/桌面/secp256k1

nl -ba CHANGELOG.md | sed -n '139,146p'
139  ## [0.3.1] - 2023-04-10
140  We strongly recommend updating to 0.3.1 if you use or plan to use Clang >=14 to compile libsecp256k1, e.
g., Xcode >=14 on macOS has Clang >=14. When in doubt, check the Clang version using `clang -v`.
141
142  #### Security
143  - Fix "constant-timeness" issue with Clang >=14 that could leave applications using libsecp256k1 vulner
able to a timing side-channel attack. The fix avoids secret-dependent control flow and secret-dependent memory a
ccesses in conditional moves of memory objects when libsecp256k1 is compiled with Clang >=14.
144
145  #### Added
146  - Added tests against [Project Wycheproof's](https://github.com/C2SP/wycheproof/) set of ECDSA test ve
ctors (Bitcoin "low-S" variant), a fixed set of test cases designed to trigger various edge cases.
```

图 5.5 libsecp256k1 v0.3.1 的常数时间安全修复说明

更新记录指出，当使用 Clang 14 及以上版本编译时，条件移动函数可能产生秘密相关控制流和秘密相关内存访问，使使用该库的程序面临时序侧信道风险。

相关修改位于提交：

4a496a3

该提交的完整说明如图 5.6 所示。

```
commit 4a496a36fb07d6cc8c99e591994f4ce0c3b1174c
Author: Tim Ruffing <crypto@timruffing.de>
Date: Sat Apr 1 15:35:50 2023 +0900

ct: Use volatile "trick" in all fe/scalar cmov implementations

Apparently clang 15 is able to compile our cmov code into a branch,
at least for fe_cmov and fe_storage_cmov. This commit makes the
condition volatile in all cmov implementations (except ge but that
one only calls into the fe impls).

This is just a quick fix. We should still look into other methods,
e.g., asm and #457. We should also consider not caring about
constant-time in scalar_low_impl.h

We should also consider testing on very new compilers in nightly CI,
see https://github.com/bitcoin-core/secp256k1/pull/864#issuecomment-769211867

src/field_10x26_impl.h | 6 +++--
src/field_5x52_impl.h | 6 +++--
src/scalar_4x64_impl.h | 3 ++-
src/scalar_8x32_impl.h | 3 ++-
src/scalar_low_impl.h | 3 ++-
5 files changed, 14 insertions(+), 7 deletions(-)
```

图 5.6 常数时间修复提交的说明及修改范围

提交说明指出, Clang 15 能够将部分条件移动代码编译为条件分支, 至少会影响:

- (1) secp256k1_fe_cmov;
- (2) secp256k1_fe_storage_cmov。

条件移动函数需要根据 `flag` 决定保留原值还是复制另一个值。其逻辑为:

$$r \leftarrow \begin{cases} r, & \text{flag} = 0, \\ a, & \text{flag} = 1. \end{cases}$$

为了避免显式条件分支, 原实现使用掩码完成选择:

```
1 mask0 = flag + ~((uint64_t)0);
2 mask1 = ~mask0;
3
4 r->n[0] =
5     (r->n[0] & mask0) |
6     (a->n[0] & mask1);
```

当 `flag=0` 时:

$$\text{mask}_0 = 2^{64} - 1, \quad \text{mask}_1 = 0,$$

所以输出保留原来的 r 。

当 `flag=1` 时, 无符号整数加法按模 2^{64} 回绕:

$$1 + (2^{64} - 1) \equiv 0 \pmod{2^{64}},$$

因此：

$$mask_0 = 0, \quad mask_1 = 2^{64} - 1,$$

输出变为 a 。

虽然 C 源码没有使用 if，但编译器可以识别出该表达式实际上是在两个值之间进行选择，并将其重新生成条件跳转或条件内存访问。如果 flag 与秘密状态有关，不同分支路径、缓存访问或分支预测行为可能泄露信息。

修复前后的代码差异如图 5.7 所示。

```
diff --git a/src/field_5x52_impl.h b/src/field_5x52_impl.h
index 4c4466e..6b97157 100644
--- a/src/field_5x52_impl.h
+++ b/src/field_5x52_impl.h
@@ -487,8 +487,9 @@ static void secp256k1_fe_sqr(secp256k1_fe *r, const secp256k1_fe *a) {

static SECP256K1_INLINE void secp256k1_fe_cmov(secp256k1_fe *r, const secp256k1_fe *a, int flag) {
    uint64_t mask0, mask1;
+   volatile int vflag = flag;
    SECP256K1_CHECKMEM_CHECK_VERIFY(r->n, sizeof(r->n));
-   mask0 = flag + ~((uint64_t)0);
+   mask0 = vflag + ~((uint64_t)0);
+   mask0 = vflag + ~((uint64_t)0);
    mask1 = ~mask0;
    r->n[0] = (r->n[0] & mask0) | (a->n[0] & mask1);
    r->n[1] = (r->n[1] & mask0) | (a->n[1] & mask1);
@@ -570,8 +571,9 @@ static SECP256K1_INLINE void secp256k1_fe_half(secp256k1_fe *r) {

static SECP256K1_INLINE void secp256k1_fe_storage_cmov(secp256k1_fe_storage *r, const secp256k1_fe_storage *a, int flag) {
```

图 5.7 条件移动函数中的 volatile 修复

修复增加了：

```
1 volatile int vflag = flag;
```

并将：

```
1 mask0 = flag + ~((uint64_t)0);
```

修改为：

```
1 mask0 = vflag + ~((uint64_t)0);
```

在受影响的 Clang 版本中，读取 volatile 变量属于必须保留的可观察操作，编译器不能像处理普通局部变量一样任意消除或合并该读取。该修改用于阻止编译器将原掩码选择重新转换成条件分支或条件内存读取。

提交同时修改了：

- (1) field_10x26_impl.h;
- (2) field_5x52_impl.h;
- (3) scalar_4x64_impl.h;
- (4) scalar_8x32_impl.h;
- (5) scalar_low_impl.h。

这些文件对应不同的有限域表示、标量表示和平台配置。若只修改其中一个实现，其他配置仍可能出现相同的编译结果，因此该提交对相关条件移动函数进行了统一处理。

`volatile` 在这里是针对特定编译器行为采取的修复方法，不能据此认为任意使用 `volatile` 的程序都具有常数时间性质。密码库仍需在具体编译器和目标平台上检查生成的机器代码，并配合常数时间测试验证控制流和内存访问是否与秘密数据无关。

6 ECDSA 验证性能改进实验

本节比较 v0.4.0 与 v0.4.1 两个版本的 ECDSA 验证性能。两个版本在同一虚拟机中使用相同编译器、编译参数和官方基准测试程序进行编译与测试，以考察版本更新后 `secp256k1_ecdsa_verify` 的耗时变化。

6.1 性能改进内容与实验设计

`libsecp256k1` v0.4.1 的更新记录如图 6.1 所示。该版本移除了有限域运算中使用的手写 x86-64 汇编实现，原因是现代 C 编译器已经能够根据 C 语言实现生成效率更高的机器代码。



```
92 ## [0.4.1] - 2023-12-21
93
94 #### Changed
95 - The point multiplication algorithm used for ECDH operations (module `ecdh`) was replaced with a slightly faster one
96 - Optional handwritten x86_64 assembly for field operations was removed because modern C compilers are able to output more efficient assembly. This change results in a significant speedup of some library functions when handwritten x86_64 assembly is enabled (`--with-asm=x86_64` in GNU Autotools, `-DSECP256K1_ASM=x86_64` in CMake), which is the default on x86_64. Benchmarks with GCC 10.5.0 show a 10% speedup for `secp256k1_ecdsa_verify` and `secp256k1_schnorrsg_verify`.
```

图 6.1 `libsecp256k1` v0.4.1 的性能改进说明

为进一步确认更新记录所对应的实际源码修改，本实验追踪了 v0.4.0 至 v0.4.1 之间的 Git 提交。真正实施该性能修改的提交为：

```
2f0762f: field: Remove x86_64 asm
```

该提交的说明与修改规模如图 6.2 所示。

```
==== Performance commit ====
2f0762fa8fd30b457bc5dcf53403123212091df5
Author: Tim Ruffing
Date: 2023-11-23

field: Remove x86_64 asm

Widely available versions of GCC and Clang beat our field asm on -O2.
In particular, GCC 10.5.0, which is Bitcoin Core's current compiler
for official x86_64 builds, produces code that is > 20% faster for
fe_mul and > 10% faster for signature verification (see #726).

These are the alternatives to this PR:

==== Files changed ====
Makefile.am          | 1 -
README.md            | 2 +-
src/field_5x52_asm_impl.h | 504 -----
src/field_5x52_impl.h   | 4 -
4 files changed, 1 insertion(+), 510 deletions(-)
delete mode 100644 src/field_5x52_asm_impl.h
```

图 6.2 移除有限域 x86-64 汇编实现的提交说明与修改规模

提交说明指出，在 -O2 优化条件下，常见版本的 GCC 和 Clang 为有限域 C 实现生成的机器代码已经快于原有的手写汇编。其中，Bitcoin Core 当时用于官方 x86-64 构建的 GCC 10.5.0 生成的有限域乘法 `fe_mul` 代码快 20% 以上，签名验证速度提高 10% 以上。

从文件修改规模看，该提交删除了：

`src/field_5x52_asm_impl.h`

该文件原本包含 504 行有限域 x86-64 汇编实现。提交同时修改了 `Makefile.am`、`README.md` 和 `src/field_5x52_impl.h`，总计删除 510 行代码。因此，此次性能变化并非 benchmark 参数调整造成，而是对应了实际的底层有限域实现修改。

相关构建文件和源码选择逻辑的差异如图 6.3 所示。

```
@@ -37,7 +37,6 @@ noinst_HEADERS += src/field_10x26_impl.h
noinst_HEADERS += src/field_5x52.h
noinst_HEADERS += src/field_5x52_impl.h
noinst_HEADERS += src/field_5x52_int128_impl.h
-noinst_HEADERS += src/field_5x52_asm_impl.h
noinst_HEADERS += src/modinv32.h
noinst_HEADERS += src/modinv32_impl.h
noinst_HEADERS += src/modinv64.h
diff --git a/README.md b/README.md
index 8c32507..1b6eedd 100644
--
    * Expose only higher level interfaces to minimize the API surface and improve application security. ("Be difficult to use in
    securely.")
    * Field operations
    * Optimized implementation of arithmetic modulo the curve's field size ( $2^{256} - 0x1000003D1$ ).
-   * Using 5 52-bit limbs (including hand-optimized assembly for x86_64, by Diederik Huys).
+   * Using 5 52-bit limbs
    * Using 10 26-bit limbs (including hand-optimized assembly for 32-bit ARM, by Wladimir J. van der Laan).
    * This is an experimental feature that has not received enough scrutiny to satisfy the standard of quality of this libra
ry but is made available for testing and review by the community.
    * Scalar operations
diff --git a/src/field_5x52_impl.h b/src/field_5x52_impl.h
index ecb7050..76031f7 100644
--- a/src/field_5x52_impl.h

```

图 6.3 有限域汇编文件移除及实现选择逻辑的源码变化

在 Makefile.am 中,src/field_5x52_asm_impl.h 不再被列入构建所使用的头文件;在 README.md 中,5 个 52 位 limb 实现的说明也删除了手写 x86-64 汇编相关内容。

更关键的变化出现在 `src/field_5x52_impl.h` 中。旧版本根据宏 `USE_ASM_X86_64` 在两种实现之间进行选择：

```
1 #if defined(USE_ASM_X86_64)
2 #include "field_5x52_asm_impl.h"
3 #else
4 #include "field_5x52_int128_impl.h"
5 #endif
```

修改后，上述条件选择被移除，源码直接包含：

```
1 #include "field_5x52_int128_impl.h"
```

这表明在使用 5 个 52 位 limb 的有限域表示时，程序不再进入原有 x86-64 手写汇编路径，而是统一使用基于 128 位整数运算的 C 实现，并由编译器在 -O2 条件下生成最终机器代码。

需要注意的是，该提交并没有删除库中的全部汇编代码。提交说明明确指出，标量运算的汇编实现仍具有性能优势。若直接关闭整个汇编构建选项，也会连同仍然较快的标量汇编一起禁用。因此，开发者最终只移除了已经落后于编译器输出的有限域 x86-64 汇编路径，而没有简单地关闭所有汇编实现。

为了验证该更新对 ECDSA 验证性能的影响, 本实验分别检出:

- ```
(1) libsecp256k1 v0.4.0;
```

(2) libsecp256k1 v0.4.1。

两个版本使用 `git worktree` 放置在不同目录中：

```
1 git worktree add ~/桌面/secp-v040 v0.4.0
2 git worktree add ~/桌面/secp-v041 v0.4.1
```

采用独立目录可以避免两个版本共用配置缓存、目标文件或可执行程序。两个版本均使用以下方式配置和编译：

```
1 ./autogen.sh
2
3 CC=gcc CFLAGS="-O2" ./configure \
4 --enable-benchmark
5
6 make -j"${nproc}"
```

实验控制条件如表 6.1 所示。

表 6.1 性能对比实验的控制条件

| 项目     | 设置                     |
|--------|------------------------|
| 运行环境   | 同一 Ubuntu 虚拟机          |
| 处理器架构  | x86-64                 |
| 编译器    | GCC                    |
| 编译优化   | -O2                    |
| 测试程序   | 官方 bench               |
| 预热次数   | 每个版本 1 次               |
| 正式测试次数 | 每个版本 5 次               |
| 记录指标   | ecdsa_verify 的 Avg(us) |

第一次运行只用于预热，不计入正式结果：

```
1 ./bench > /dev/null
```

随后连续运行五次官方 benchmark，并将完整输出保存至文本文件：

```
1 for i in 1 2 3 4 5
2 do
3 echo "===== Run $i ====="
4 ./bench
5 echo
6 done | tee bench_5runs.txt
```

测试过程中仅提取 `ecdsa_verify` 对应的结果。官方 `bench` 输出的三项数据依次为最小耗时、平均耗时和最大耗时，本实验使用中间的 `Avg(us)` 作为比较指标。

## 6.2 测试结果与性能计算

v0.4.0 五轮 ECDSA 验证结果如图 6.4 所示。



```
openhitals@1111:~/桌面/secp-v040$ grep 'ecdsa_verify' \
~/桌面/ecdsa-lab/results/performance/bench_v040_5runs.txt
ecdsa_verify , 32.1 , 33.1 , 34.6
ecdsa_verify , 31.5 , 32.1 , 32.7
ecdsa_verify , 31.5 , 32.0 , 32.4
ecdsa_verify , 31.5 , 32.0 , 32.4
ecdsa_verify , 31.0 , 31.4 , 31.9
```

图 6.4 libsecp256k1 v0.4.0 的五轮 ECDSA 验证测试结果

五轮测试的平均耗时分别为：

33.1, 32.1, 32.0, 32.0, 31.4  $\mu s$ .

v0.4.1 五轮测试结果如图 6.5 所示。

```
openhitals@1111:~/桌面/secp-v041$ grep 'ecdsa_verify' \
~/桌面/ecdsa-lab/results/performance/bench_v041_5runs.txt
ecdsa_verify , 27.5 , 28.3 , 28.8
ecdsa_verify , 28.0 , 28.6 , 29.0
ecdsa_verify , 28.3 , 28.6 , 29.1
ecdsa_verify , 28.2 , 28.6 , 29.2
ecdsa_verify , 27.4 , 27.9 , 29.2
```

图 6.5 libsecp256k1 v0.4.1 的五轮 ECDSA 验证测试结果

五轮测试的平均耗时分别为：

28.3, 28.6, 28.6, 28.6, 27.9  $\mu s$ .

原始数据整理如表 6.2 所示。

表 6.2 v0.4.0 与 v0.4.1 的 ECDSA 验证耗时

| 测试轮次  | v0.4.0 Avg/ $\mu s$ | v0.4.1 Avg/ $\mu s$ |
|-------|---------------------|---------------------|
| 第 1 次 | 33.1                | 28.3                |
| 第 2 次 | 32.1                | 28.6                |
| 第 3 次 | 32.0                | 28.6                |
| 第 4 次 | 32.0                | 28.6                |
| 第 5 次 | 31.4                | 27.9                |
| 五轮均值  | 32.12               | 28.40               |

v0.4.0 的总体平均耗时为：

$$\begin{aligned}
 T_{0.4.0} &= \frac{33.1 + 32.1 + 32.0 + 32.0 + 31.4}{5} \\
 &= 32.12 \mu s.
 \end{aligned}$$

v0.4.1 的总体平均耗时为：

$$T_{0.4.1} = \frac{28.3 + 28.6 + 28.6 + 28.6 + 27.9}{5} \\ = 28.40 \mu s.$$

采用平均耗时下降比例比较两个版本：

$$\text{Reduction} = \frac{T_{0.4.0} - T_{0.4.1}}{T_{0.4.0}} \times 100\%.$$

代入测试结果可得：

$$\text{Reduction} = \frac{32.12 - 28.40}{32.12} \times 100\% \\ \approx 11.58\%.$$

最终统计结果如图 6.6 所示。

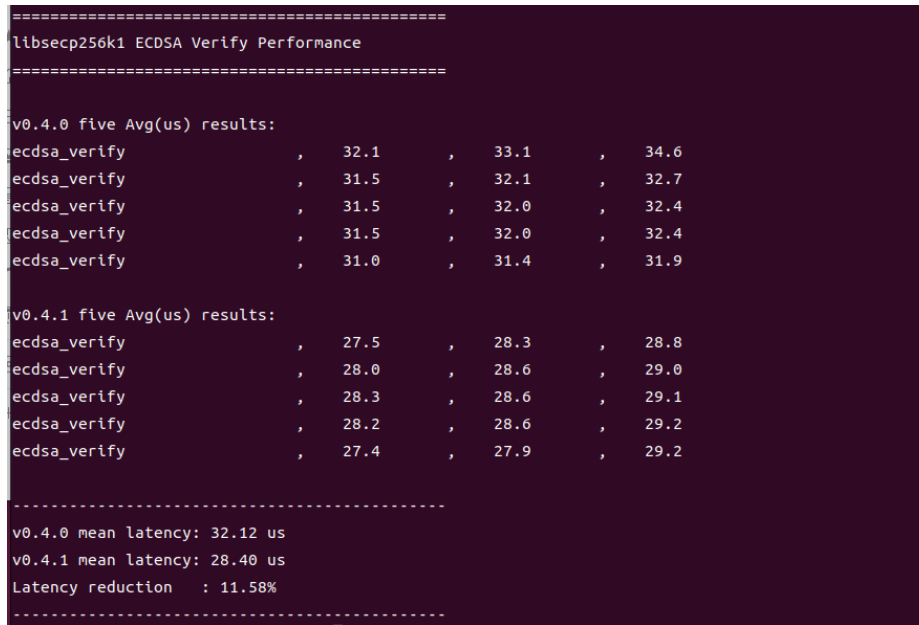


图 6.6 两个版本的 ECDSA 验证性能对比结果

在当前实验环境中，v0.4.1 的平均验证耗时由  $32.12 \mu s$  降至  $28.40 \mu s$ ，单次验证平均减少：

$$32.12 - 28.40 = 3.72 \mu s.$$

平均耗时下降约 11.58%。若按照单位时间内可完成的验证次数计算，速度提高比例为：

$$\text{Speedup} = \left( \frac{T_{0.4.0}}{T_{0.4.1}} - 1 \right) \times 100\% \\ = \left( \frac{32.12}{28.40} - 1 \right) \times 100\% \\ \approx 13.10\%.$$

官方更新记录报告约 10% 的 speedup，本实验得到的速度提高约 13.10%，二者处于相近范围。处

理器、GCC 版本、虚拟机调度和测试方法不同，均可能使具体数值产生差异。

### 6.3 性能变化原因及实验局限

ECDSA 验证首先计算：

$$w = s^{-1} \bmod n, \quad u_1 = ew \bmod n, \quad u_2 = rw \bmod n,$$

随后计算椭圆曲线上的双标量乘法：

$$R = u_1G + u_2P.$$

双标量乘法并不是一次普通整数乘法，而是由多轮点加和倍点运算构成。为了避免频繁执行有限域求逆，实际实现通常使用 Jacobian 坐标表示中间点。在这种坐标表示下，点加和倍点会反复调用有限域乘法、平方、加减和约减等底层运算。

提交 2f0762f 重点涉及的 `fe_mul` 即有限域元素乘法。该运算在一次 ECDSA 验证的双标量乘法过程中会被多次调用，因此其单次耗时降低后，性能收益可以在多轮点运算中累积，并最终反映为 `secp256k1_ecdsa_verify` 总体耗时的下降。

不过，有限域乘法并不是 ECDSA 验证的全部开销。验证过程还包括标量求逆和乘法、预计算表访问、有限域平方、点加和倍点、无穷远点判断以及最终横坐标比较。因此，`fe_mul` 快 20% 以上，并不意味着整个 ECDSA 验证也必然提高 20%。官方提交中报告的签名验证性能提高超过 10%，低于单独有限域乘法的提升比例，符合底层热点运算只占整体耗时一部分的情况。

本实验测得 v0.4.1 的平均验证耗时由  $32.12 \mu s$  降至  $28.40 \mu s$ ，平均耗时下降 11.58%，按单位时间内能够完成的验证次数计算，速度提高 13.10%。实验结果与官方提交所报告的签名验证性能提升超过 10% 处于相近范围。

从源码修改可以看出，性能变化并不是由 ECDSA 数学公式发生变化造成的。两个版本仍然执行相同的验证关系：

$$R = s^{-1}eG + s^{-1}rP.$$

变化发生在该计算所依赖的底层有限域实现中：旧版本在满足 `USE_ASM_X86_64` 条件时使用手写汇编，新版本则统一使用 `field_5x52_int128_impl.h` 中的实现，并由现代编译器生成最终机器代码。也就是说，版本更新保持了算法和计算结果不变，只改变了完成相同有限域运算的工程实现方式。

手写汇编并不天然比编译器生成的代码更快。手工实现通常针对编写时的处理器和编译器环境进行优化；随着编译器的寄存器分配、指令选择和指令调度能力提高，较早编写的汇编实现可能逐渐失去优势。此外，保留内联汇编还会增加代码维护、跨平台测试和人工审查的成本。此次修改在提高当前平台性能的同时，删除了 504 行专用汇编代码，也降低了实现复杂度。

本实验没有进一步对两个版本生成的机器代码进行逐指令比较，也没有使用硬件性能计数器统计指令周期、分支预测、缓存未命中或流水线停顿。因此，报告不能将全部耗时差异归因于某一条具体指令，只能根据官方提交说明、源码差异和相同条件下的 benchmark 结果，判断本次性能变化与有限域实现路径的修改一致。

此外，本实验还存在以下限制：

- (1) 测试运行于虚拟机，结果可能受到宿主机调度和其他进程负载影响；

- (2) 未固定 CPU 工作频率, 动态调频可能造成小幅时间波动;
- (3) 仅使用 GCC 和 -O2 编译参数, 没有比较 Clang、其他 GCC 版本或不同优化级别;
- (4) 每个版本正式测试五次, 能够观察总体趋势, 但样本数量仍然有限;
- (5) 未在不同处理器架构或物理机环境中重复实验;
- (6) 两个版本按照固定顺序完成测试, 没有采用交替顺序或反向顺序复测, 系统状态随时间变化可能对结果产生一定影响。

尽管存在上述限制, 两个版本的五轮测试结果没有出现重叠: v0.4.0 的最低平均耗时为  $31.4 \mu s$ , 仍高于 v0.4.1 的最高平均耗时  $28.6 \mu s$ 。同时, 实测结果与官方提交中报告的性能变化方向和幅度基本一致。因此, 在本实验环境和编译条件下, 可以确认 v0.4.1 的 ECDSA 验证性能高于 v0.4.0。

## 7 实验总结

本实验基于 Bitcoin Core 官方 libsecp256k1, 完成了 ECDSA 自选摘要签名构造、验证源码分析、常数时间安全修复分析以及不同版本的性能对比。

在构造实验中, 首先使用随机生成的私钥和公钥完成正常 ECDSA 签名, 官方验证接口返回 PASS, 说明实验环境、消息哈希、密钥生成和签名验证过程均能够正常运行。

随后, 构造函数在不接收目标私钥的条件下, 仅利用目标公钥和随机选择的标量  $u, v$ , 生成摘要  $e'$  与签名  $(r', s')$ 。该签名能够通过 secp256k1\_ecdsa\_verify 的验证, 说明课件中的代数构造可以在真实密码库中复现。

当保持签名  $(r', s')$  不变, 并将摘要替换为真实消息 Pay Alice 1 BTC 的 SHA-256 值后, 验证结果为 FAIL。这一结果说明, 构造程序得到的是一组相互匹配的摘要和签名, 而不是某条预先指定消息的有效签名。若验证者从原始消息中自行计算摘要, 该构造不能通过验证, 因此实验结果不表示 ECDSA 本身被破解。

源码分析进一步验证了上述结论。secp256k1\_ecdsa\_verify 接收的是 32 字节摘要 msghash32, 而不是原始消息。该接口只能判断摘要、签名和公钥是否满足 ECDSA 验证关系, 无法判断摘要是否确实由收到的消息计算得到。消息与摘要的一致性必须由调用该接口的上层程序负责。

内部验证函数按照以下关系计算验证点:

$$u_1 = s^{-1}e \bmod n, \quad u_2 = s^{-1}r \bmod n,$$

$$R = u_1G + u_2P.$$

构造程序通过选择  $e', r'$  和  $s'$ , 使内部计算得到的系数为  $u, v$ ; 若执行 lower-S 规范化, 则两个系数同时变为  $-u, -v$ 。对应的验证点分别为  $R'$  或  $-R'$ , 二者横坐标相同, 因此源码分析与实验结果能够相互对应。

在常数时间安全分析中, 实验定位了 libsecp256k1 v0.3.1 的相关修复。原条件移动代码虽然没有显式使用条件分支, 但 Clang 14 及以上版本可能将其重新编译为秘密相关的控制流或内存访问。修复通过引入 volatile 条件变量, 抑制该编译器优化。该问题说明, 密码程序的安全性不仅取决于 C 源码, 还取决于编译器最终生成的机器代码。

性能实验分别测试了 v0.4.0 和 v0.4.1。在相同虚拟机、GCC 编译器和 -O2 优化参数下, 两

个版本的 ECDSA 验证平均耗时分别为：

$$T_{0.4.0} = 32.12 \mu s, \quad T_{0.4.1} = 28.40 \mu s.$$

v0.4.1 的单次验证平均耗时减少了  $3.72 \mu s$ ，平均耗时下降约 11.58%；按单位时间内可完成的验证次数计算，验证速度提高约 13.10%。

源码追踪表明，真正实施该性能修改的提交为 `2f0762f`。该提交删除了包含 504 行代码的有限域 x86-64 手写汇编实现，并将 5 个 52 位 limb 的有限域运算统一切换至 int128 实现，由现代编译器生成最终机器代码。由于 ECDSA 验证中的双标量乘法会反复调用有限域乘法等底层运算，该实现变化能够累积为整体签名验证性能的提高。实验测得的性能变化与官方提交所报告的结果处于相近范围。

## 8 参考文献

- [1] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. FIPS PUB 186-5, U.S. Department of Commerce, 2023.
- [2] Certicom Research. *SEC 2: Recommended Elliptic Curve Domain Parameters*. Version 2.0, Standards for Efficient Cryptography Group, 2010.
- [3] Pornin T. *Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)*. RFC 6979, Internet Engineering Task Force, 2013.
- [4] Kocher P. C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In: Koblitz N. (ed.), *Advances in Cryptology—CRYPTO '96*, Lecture Notes in Computer Science, Vol. 1109, Springer, 1996: 104–113.
- [5] Bitcoin Core Developers. *libsecp256k1: High-performance High-assurance C Library for Cryptographic Primitives on the secp256k1 Curve*. GitHub Repository, <https://github.com/bitcoin-core/secp256k1>.